

HTTP Module and Basic Servers — HTTP Basics

Section: 6 — HTTP Module and Basic Servers

Subtopic: 01 — HTTP Basics

Estimated Duration: 3–4 hours

Theory

Practical

Topics covered: The `http` module, `createServer`, request and response objects, HTTP methods (GET, POST, PUT, DELETE), headers, status codes, content types

Learning Objectives — This Subtopic

- Create an HTTP server using the built-in `http` module without any frameworks
- Identify and describe the purpose of the `request` and `response` objects
- Distinguish between the four main HTTP methods (GET, POST, PUT, DELETE) and their intended use
- Set and read HTTP headers on both requests and responses
- Send appropriate HTTP status codes and explain the meaning of the major status code ranges
- Serve different content types (plain text, HTML, JSON) from a Node.js server

1. How the Web Works — A Quick Refresher

Before writing any server code, it is worth establishing the mental model that every HTTP interaction follows. A **client** (usually a web browser, but it can also be a mobile app, another server, or a CLI tool like `curl`) sends a **request** to a server. The server processes the request and sends back a **response**.

Every HTTP request contains at minimum:

- A **method** (what the client wants to do — e.g. GET, POST)
- A **URL** (which resource the client is addressing)

- **Headers** (metadata about the request — e.g. what format the client expects)
- An optional **body** (data sent with the request, common in POST/PUT)

Every HTTP response contains:

- A **status code** (a numeric indicator of what happened — e.g. 200 OK, 404 Not Found)
- **Headers** (metadata about the response — e.g. content type, caching instructions)
- An optional **body** (the actual data — HTML, JSON, a file, etc.)

Analogy: Think of HTTP like sending a letter. The *method* is what you want (a question, a delivery, a change-of-address form). The *URL* is the recipient's address. The *headers* are the markings on the envelope (fragile, priority, return address). The *body* is the letter inside. The response is the reply letter you receive back, with its own envelope markings and contents.

2. The Node.js `http` Module

Node.js ships with a built-in `http` module. No installation is required — it is part of the Node.js core. This module provides everything needed to create both HTTP servers and HTTP clients. ⓘ

Import it using CommonJS:

`server.js`

```
const http = require('http');
```

Note: The `http` module is a core module — it does not need to be installed via NPM. Core modules are always available in any Node.js installation.

2.1 Creating the Simplest Possible Server

The `http.createServer()` function accepts a **request handler** — a callback function that Node.js invokes every time an HTTP request arrives. The callback receives two arguments: the **request object** (`req`) and the **response object** (`res`).

Set up a project folder first:

```
mkdir http-basics && cd http-basics
```

`server.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello from Node.js');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Run the server:

```
node server.js
```

Output:

```
Server running at http://localhost:3000/
```

Open `http://localhost:3000/` in a browser. The page displays **Hello from Node.js**.

What is happening?

- `http.createServer()` returns a `Server` object. The callback passed to it is executed for every incoming request.
- `res.end()` signals that the response is complete and sends the provided string as the body. The connection is closed after this call.

- `server.listen(3000, callback)` binds the server to port 3000. The callback fires once the server is ready to accept connections.

Important: The server process keeps running until you stop it. Press `Ctrl` + `C` in the terminal to terminate the process. If you forget to call `res.end()`, the client's request will hang indefinitely.

3. The Request Object (`req`)

The `req` parameter in the request handler is an instance of `http.IncomingMessage`. It contains all the information about the incoming request. The most commonly used properties are:

Property	Type	Description
<code>req.method</code>	String	The HTTP method (GET, POST, etc.)
<code>req.url</code>	String	The request path and query string (e.g. <code>/users?page=2</code>)
<code>req.headers</code>	Object	Key-value pairs of all request headers (all keys are lowercased)
<code>req.httpVersion</code>	String	The HTTP version used (e.g. <code>1.1</code>)

Let us inspect the request object by logging its key properties:

`server.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
  console.log('--- Incoming Request ---');
  console.log('Method:', req.method);
});
```

```
console.log('URL:', req.url);
console.log('Headers:', JSON.stringify(req.headers, null, 2));

res.end('Request received');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Run the server and visit `http://localhost:3000/about?lang=en` in a browser:

Output:

```
Server running at http://localhost:3000/
--- Incoming Request ---
Method: GET
URL: /about?lang=en
Headers: {
  "host": "localhost:3000",
  "connection": "keep-alive",
  "user-agent": "Mozilla/5.0 ...",
  "accept": "text/html,application/xhtml+xml,...",
  "accept-encoding": "gzip, deflate, br",
  "accept-language": "en-GB,en;q=0.9"
}
```

What is happening?

- `req.method` is `"GET"` because the browser issues a GET request by default when navigating to a URL.
- `req.url` contains the path (`/about`) and the query string (`?lang=en`). It does *not* include the protocol or host.
- `req.headers` is a plain object. Notice that all header keys are lowercased — `User-Agent` becomes `user-agent` . This is a Node.js convention to simplify lookups.

Key Insight: Browsers often send a second request automatically for `/favicon.ico` . Do not be surprised if you see two `--- Incoming Request ---` log entries. This is normal browser behaviour.

4. The Response Object (`res`)

The `res` parameter is an instance of `http.ServerResponse`. It provides methods to construct and send the HTTP response back to the client.

4.1 Core Response Methods

Method	Purpose
<code>res.statusCode = 200</code>	Sets the HTTP status code
<code>res.setHeader(name, value)</code>	Sets a single response header
<code>res.writeHead(statusCode, headers)</code>	Sets status code and multiple headers at once
<code>res.write(data)</code>	Sends a chunk of the response body (can be called multiple times)
<code>res.end([data])</code>	Signals the response is complete. Optionally sends final data.

4.2 Setting Headers and Status Codes

Stage 1 Set the status code and content type explicitly:

`server.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('This is plain text');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

This is plain text

Stage 2 Use `writeHead()` to set status code and headers in a single call:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/html',
    'X-Powered-By': 'Node.js'
  });
  res.end('<h1>Hello from HTML</h1>');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

(Browser renders the heading)
Hello from HTML

What is happening?

- `res.writeHead(200, { ... })` sets the status code and all headers in one call. Once `writeHead` is called, headers are flushed to the client — you cannot modify them afterwards.
- The `Content-Type: text/html` header tells the browser to interpret the body as HTML rather than raw text.
- `X-Powered-By` is a custom header. Any header prefixed with `x-` is conventionally non-standard.

4.3 Using `res.write()` for Multi-Part Responses

You can call `res.write()` multiple times before calling `res.end()`. Each call appends data to the response body:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });

  res.write('<html><body>');
  res.write('<h1>Welcome</h1>');
  res.write('<p>This response was built in parts.</p>');
  res.write('</body></html>');

  res.end();
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

```
(Browser renders:)
Welcome
This response was built in parts.
```

This pattern is useful when you need to construct the response body incrementally, for example when reading from a data source chunk by chunk. `res.end()` with no argument simply closes the response after all `write()` calls are complete.

Warning: You must always call `res.end()` on every response. If you call `res.write()` without eventually calling `res.end()`, the client will wait indefinitely for more data.

5. HTTP Methods

HTTP defines several request methods, each with a specific semantic meaning. The four most common ones form the backbone of REST APIs:

Method	Purpose	Has Body?	Idempotent?
GET	Retrieve a resource	No	Yes
POST	Create a new resource / submit data	Yes	No
PUT	Replace a resource entirely	Yes	Yes
DELETE	Remove a resource	Rarely	Yes

Key Insight: *Idempotent* means that making the same request multiple times produces the same result. A GET for `/users/5` always returns the same user. A DELETE on `/users/5` results in user 5 being removed — calling it again still results in user 5 not existing. POST is not idempotent because each call may create a new resource.

5.1 Routing by Method

The built-in `http` module does not provide a routing system. You must inspect `req.method` and `req.url` yourself and respond accordingly:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  const { method, url } = req;

  if (method === 'GET' && url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home page');
  } else if (method === 'GET' && url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('About page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Not Found');
  }
});
```

```
});  
  
server.listen(3000, () => {  
  console.log('Server running at http://localhost:3000/');  
});
```

Test from the terminal using `curl` :

```
curl http://localhost:3000/
```

Output:

Home page

```
curl http://localhost:3000/about
```

Output:

About page

```
curl http://localhost:3000/contact
```

Output:

Not Found

What is happening?

- We destructure `method` and `url` from the request object for convenience.
- We check for specific method + URL combinations. Anything unrecognised falls through to the `else` block and returns a 404.
- `curl` sends GET requests by default. It is a useful tool for testing servers without opening a browser.

Note: This manual routing approach becomes unwieldy as routes grow. In Section 7, Express.js provides a much cleaner routing system — but understanding the raw approach first is essential.

6. HTTP Status Codes

Status codes are three-digit numbers grouped into five ranges. Memorising every code is unnecessary, but understanding the ranges and the most common codes is critical:

Range	Category	Common Codes
1xx	Informational	100 Continue , 101 Switching Protocols
2xx	Success	200 OK , 201 Created , 204 No Content
3xx	Redirection	301 Moved Permanently , 302 Found , 304 Not Modified
4xx	Client Error	400 Bad Request , 401 Unauthorised , 403 Forbidden , 404 Not Found
5xx	Server Error	500 Internal Server Error , 502 Bad Gateway , 503 Service Unavailable

6.1 Applying Status Codes

Let us extend our server to use semantically correct status codes:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  const { method, url } = req;

  if (method === 'GET' && url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
```

```
res.end('Home page');

} else if (method === 'GET' && url === '/old-page') {
  // Permanent redirect to new location
  res.writeHead(301, { 'Location': '/new-page' });
  res.end();

} else if (method === 'GET' && url === '/new-page') {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('This is the new page');

} else if (method === 'GET' && url === '/error') {
  res.writeHead(500, { 'Content-Type': 'text/plain' });
  res.end('Internal Server Error');

} else {
  res.writeHead(404, { 'Content-Type': 'text/plain' });
  res.end('404 Not Found');
}
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Test the redirect:

```
curl -v http://localhost:3000/old-page
```

Output:

```
< HTTP/1.1 301 Moved Permanently
< Location: /new-page
```

```
curl -L http://localhost:3000/old-page
```

Output:

```
This is the new page
```

What is happening?

- The `301` status code tells the client the resource has permanently moved. The `Location` header specifies where.
- `curl -v` shows verbose output including response headers. `curl -L` follows redirects automatically.
- Browsers follow redirects automatically — visiting `/old-page` in a browser redirects the user to `/new-page` seamlessly.

7. Content Types

The `Content-Type` header tells the client how to interpret the response body. Without it, the client has to guess — and guessing leads to bugs.

Content-Type Value	Use For
<code>text/plain</code>	Plain text, no formatting
<code>text/html</code>	HTML documents
<code>application/json</code>	JSON data (the standard for APIs)
<code>text/css</code>	CSS stylesheets
<code>application/javascript</code>	JavaScript files
<code>image/png</code> , <code>image/jpeg</code>	Images

7.1 Serving Different Content Types

Stage 1 Serve plain text:

`server.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
```

```
res.writeHead(200, { 'Content-Type': 'text/plain' });
res.end('This is plain text. HTML tags like <b>bold</b> will NOT render.');
```

```
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

This is plain text. HTML tags like bold will NOT render.

Stage 2 Serve HTML:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>Welcome</h1><p>This is <strong>HTML</strong> content.</p>');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

(Browser renders:)
Welcome
This is **HTML** content.

Stage 3 Serve JSON (the most common format for APIs):

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  const data = {
```

```
    name: 'Alice',
    role: 'Developer',
    skills: ['Node.js', 'JavaScript', 'SQL']
  });

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(data));
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

```
curl http://localhost:3000/
```

Output:

```
{"name": "Alice", "role": "Developer", "skills": ["Node.js", "JavaScript", "SQL"]}
```

What is happening?

- `JSON.stringify()` converts a JavaScript object into a JSON string. The `res.end()` method only accepts strings or Buffers, so this conversion is required.
- Setting `Content-Type` to `application/json` is essential. Without it, the client may misinterpret the response as plain text.

Important: Always match the `Content-Type` header to the actual content you are sending. Declaring `text/html` but sending JSON (or vice versa) causes parsing errors on the client side.

8. Working with Headers

8.1 Reading Request Headers

Request headers are available as the `req.headers` object. All keys are lowercased by Node.js:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  const userAgent = req.headers['user-agent'];
  const acceptLanguage = req.headers['accept-language'] || 'Not specified';

  const info = `User-Agent: ${userAgent}\nAccept-Language: ${acceptLanguage}`;

  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end(info);
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

curl http://localhost:3000/**Output:**

```
User-Agent: curl/8.4.0
Accept-Language: Not specified
```

8.2 Setting Response Headers

There are two approaches to setting response headers:

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Approach 1: Set headers individually
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('X-Request-Id', 'abc-123');

  // You can read back a header you've set
  console.log('Content-Type set to:', res.getHeader('Content-Type'));

  // You can remove a header before sending
```



```
res.removeHeader('X-Request-Id');

res.statusCode = 200;
res.end(JSON.stringify({ message: 'Headers demonstration' }));
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

```
curl -v http://localhost:3000/
```

Output:

```
(Terminal output, server side:)
Content-Type set to: application/json

(curl verbose output, client side:)
< HTTP/1.1 200 OK
< Content-Type: application/json
< Cache-Control: no-cache
{"message":"Headers demonstration"}
```

What is happening?

- `res.setHeader()` sets individual headers and can be called multiple times. Headers are not sent until `res.writeHead()`, `res.write()`, or `res.end()` is called.
- `res.getHeader()` returns the value of a header that has been set but not yet sent.
- `res.removeHeader()` removes a header before the response is flushed. Notice `X-Request-Id` does not appear in the output because it was removed before sending.

Note: `res.setHeader()` and `res.writeHead()` serve different purposes. Use `setHeader()` when you want to build headers incrementally. Use `writeHead()` when you want to set the status code and all headers in one go. If you call `writeHead()`, any headers previously set with `setHeader()` are merged in.

9. Putting It All Together — A Multi-Route Server

Now let us combine everything into a server that handles multiple routes and content types. This example demonstrates how all the concepts covered in this subtopic work in concert:

server.js

```
const http = require('http');

const users = [
  { id: 1, name: 'Alice', role: 'Developer' },
  { id: 2, name: 'Bob', role: 'Designer' },
  { id: 3, name: 'Charlie', role: 'Manager' }
];

const server = http.createServer((req, res) => {
  const { method, url } = req;

  // GET / - HTML home page
  if (method === 'GET' && url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end(`
      <html>
      <body>
        <h1>Team Directory</h1>
        <p>Visit <a href="/api/users">/api/users</a> for the JSON API.</p>
      </body>
      </html>
    `);

    // GET /api/users - JSON list of users
  } else if (method === 'GET' && url === '/api/users') {
    res.writeHead(200, {
      'Content-Type': 'application/json',
      'X-Total-Count': users.length.toString()
    });
    res.end(JSON.stringify(users));

    // GET /health - plain text health check
  } else if (method === 'GET' && url === '/health') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('OK');

    // Anything else - 404
  } else {

```

```
res.writeHead(404, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({
  error: 'Not Found',
  message: `${method} ${url} does not exist`,
  statusCode: 404
})));
}
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Test each route:

```
curl http://localhost:3000/api/users
```

Output:

```
[{"id":1,"name":"Alice","role":"Developer"}, {"id":2,"name":"Bob","role":"Designer"}, {"id":3,"name":"Charlie","role":"Manager"}]
```

```
curl http://localhost:3000/health
```

Output:

OK

```
curl http://localhost:3000/nonexistent
```

Output:

```
{"error":"Not Found","message":"GET /nonexistent does not exist","statusCode":404}
```

What is happening?

- The server handles three routes, each with a different content type: HTML for the home page, JSON for the API endpoint, and plain text for the health check.

- The `x-Total-Count` custom header on the `/api/users` response is a common API pattern — it communicates metadata without cluttering the response body.
- The 404 handler returns a JSON error object. This is a professional pattern: even error responses should be structured and informative, not just a raw string.

Analogy: This if/else chain is like a receptionist at a building entrance. They check who you are asking for (the URL), what you need (the method), and either direct you to the right office or politely tell you nobody by that name works here (404). Express.js, which you will cover in Section 7, replaces this receptionist with an automated routing system.

Summary

Concept	Key Point
<code>http</code> module	Built-in core module — no installation needed. Provides <code>createServer()</code> for building servers.
<code>createServer(callback)</code>	Creates a server. The callback receives <code>req</code> (IncomingMessage) and <code>res</code> (ServerResponse) for every request.
Request object (<code>req</code>)	Contains <code>method</code> , <code>url</code> , <code>headers</code> , and the incoming data stream.
Response object (<code>res</code>)	Use <code>writeHead()</code> , <code>setHeader()</code> , <code>write()</code> , and <code>end()</code> to build and send the response.
HTTP methods	GET (retrieve), POST (create), PUT (replace), DELETE (remove). Check via <code>req.method</code> .
Status codes	2xx = success, 3xx = redirect, 4xx = client error, 5xx = server error. Set via <code>res.statusCode</code> or <code>writeHead()</code> .

Concept	Key Point
Content-Type header	Tells the client how to interpret the body. Common values: <code>text/plain</code> , <code>text/html</code> , <code>application/json</code> .
Headers	Read request headers from <code>req.headers</code> (lowercased keys). Set response headers with <code>setHeader()</code> or <code>writeHead()</code> .

Interview Questions

1. What does `http.createServer()` return, and what arguments does its callback receive?

Expected answer: It returns an `http.Server` instance. The callback receives two arguments: `req` (an `http.IncomingMessage` object representing the request) and `res` (an `http.ServerResponse` object used to construct and send the response).

2. What happens if you never call `res.end()` in your request handler?

Expected answer: The client's connection stays open indefinitely, waiting for the server to finish sending data. The request eventually times out on the client side.

3. What is the difference between `res.setHeader()` and `res.writeHead()` ?

Expected answer: `res.setHeader(name, value)` sets a single header and can be called multiple times before the response is sent. `res.writeHead(statusCode, headers)` sets the status code and headers all at once and flushes them — headers cannot be modified after calling `writeHead()` .

4. Why are all keys in `req.headers` lowercased?

Expected answer: HTTP headers are case-insensitive by specification. Node.js lowercases all header keys to provide a consistent, predictable lookup without case-matching issues (e.g. `req.headers['content-type']` always works).

5. Explain what the **Content-Type** header does and why it matters.

Expected answer: It tells the client the MIME type of the response body so the client knows how to parse it. For example, `application/json` means the body is JSON, `text/html` means it is an HTML document. Without it, the client has to guess, which can cause incorrect rendering or parsing errors.

6. What is the difference between a 301 and a 302 redirect?

Expected answer: A `301 Moved Permanently` indicates the resource has permanently moved and clients/search engines should update their references. A `302 Found` is a temporary redirect — the original URL may be valid again in the future.

7. What does **idempotent** mean in the context of HTTP methods? Which methods are idempotent?

Expected answer: An idempotent operation produces the same outcome regardless of how many times it is executed. GET, PUT, and DELETE are idempotent. POST is not — each POST may create a new resource.

8. How would you send a JSON error response with a 400 status code?

Expected answer: Set the status code to 400, set `Content-Type` to `application/json`, and call `res.end()` with a `JSON.stringify()` 'd error object. For example: `res.writeHead(400, { 'Content-Type': 'application/json' }); res.end(JSON.stringify({ error: 'Bad Request' }));`

Practical Assignment: Build a Mini REST API

Objective: Build an HTTP server that serves as a simple product catalogue API using only the built-in `http` module.

Requirements:

1. Create a project folder called `product-api` and initialise it with `npm init -y`.
2. Create a file called `server.js` that starts an HTTP server on port `3000`.

3. Define an array of at least 3 product objects, each with `id`, `name`, `price`, and `inStock` (boolean) properties.
4. Implement the following routes:
 - `GET /` — Returns an HTML welcome page with a link to `/api/products`.
 - `GET /api/products` — Returns the full product list as JSON with a `200` status code. Include an `X-Total-Count` header with the number of products.
 - `GET /health` — Returns plain text `OK` with a `200` status code.
 - Any other route — Returns a JSON error object with a `404` status code. The error object should include `error`, `message`, and `statusCode` fields.
5. Every response must include the correct `Content-Type` header.

Test your server with these commands:

```
curl http://localhost:3000/api/products
```

Output:

```
[{"id":1,"name":"Mechanical Keyboard","price":89.99,"inStock":true}, {"id":2,"name":"USB-C Hub","price":34.50,"inStock":true}, {"id":3,"name":"Monitor Stand","price":55.00,"inStock":false}]
```

```
curl -v http://localhost:3000/api/products 2>&1 | grep X-Total-Count
```

Output:

```
< X-Total-Count: 3
```

```
curl http://localhost:3000/unknown
```

Output:

```
{"error":"Not Found","message":"GET /unknown does not exist","statusCode":404}
```

Bonus Challenge

Add a `GET /api/products?inStock=true` route that filters the product list to only return products where `inStock` is `true`. You will need to parse the query string from `req.url`. Hint: the built-in `url` module (covered later in this section) can help, but you can also use string methods or the `URL` constructor:

Hint

```
const parsedUrl = new URL(req.url, `http://${req.headers.host}`);  
const inStockFilter = parsedUrl.searchParams.get('inStock');
```

